# Payout Rails Partner Onboarding Playbook

How I would take a new payout partner from API review to a stable live corridor, and keep it there.

## At a glance: onboarding a payout partner

Review 1–2 → Map 3 → Specify 4 → Certify 5 → Launch 6 → Run 7

**One owner per connection, into production.**

### 1 🔍 Partner API review

Status model · limits · cutoffs

× The spec says "real-time", but credits land in batches hours later.

✓ Review the spec and sandbox against a fixed question list: status model, idempotency, callbacks, limits, cutoffs and error codes.

🏁 **Done when:** The partner confirms the status model in writing, and every gap has an owner.

### 2 🔐 Environments and credential exchange

OAuth 2.0 · mTLS · JWT · API keys

× UAT blocked for a week by a certificate with the wrong chain, and a secret shared over email.

✓ Exchange certificates and secrets over an agreed secure channel, allowlist IPs, and log every expiry date on day one.

🏁 **Done when:** A sandbox call passes with mTLS and OAuth, and every expiry date has an alert.

### 3 🗺️ Corridor and rail configuration

Field mappings · alias validation · error codes

× Partner code `E17` maps to "generic failure", so nobody knows whether to retry, refund or fix.

✓ Map every field and validation rule per rail, and every partner code to a canonical code that is final, retryable, hold or unknown.

🏁 **Done when:** Every partner error code is mapped, and unmapped codes raise an alert. See the mapping.

### 4 📐 Integration spec

Sequence diagrams · test matrix

× Engineering builds from the partner's PDF, and three people read it three ways.

✓ One spec per corridor: sequence diagram, field mappings, validation rules and the test matrix. See the corridor blueprint.

🏁 **Done when:** The partner and our platform team both sign off the spec.

5

## 🧪 Certification

Raw traces as evidence

× Only the happy path was tested, and the first production timeout paid out twice.

✓ Run the full test matrix: timeouts, duplicates, rejects, callbacks, cutoffs and signature failures, with the raw request and response kept for each case.

🏁 **Done when:** Every case in the test matrix passes with its traces, and the partner certifies.

6

## 🚀 Launch

Compliance · Commercial sign-off

Controlled ramp

× Full volume on day one, and one mapping bug reaches thousands of payouts.

✓ Compliance and Commercial sign off, then a ramp: limited volume first, with live monitoring and both on-call contacts confirmed.

🏁 **Done when:** The ramp completes with every SLA at target.

7

## 📈 Run, escalate and improve

SLAs · rejection rates · roadmap

× Each failure is handled as a one-off ticket, and the same problem returns every week.

✓ A weekly review from transaction data, evidence-based escalations to the partner, and recurring causes fixed in the platform.

Proposed targets · to agree in the partner SLA

✅

**Success rate** PROPOSED

By corridor and rail, with an alert on a drop against the weekly baseline.

⛔

**Rejection rate** PROPOSED

By reason code, so a spike points at one cause, not "failures went up".

⏱️

**Time to credit** PROPOSED

p95 against the partner's SLA, split by instant and batch rails.

❓

**Unmapped codes** PROPOSED

Target zero; each new code is mapped before the next weekly review.

🏁 **Done when:** Every recurring failure has an owner: a partner fix, a platform fix or a roadmap item.

---

## Delivery plan PROPOSED ›

8 weeks for one partner and one corridor; Compliance review runs alongside the build.

A typical 8-week path for one partner and one corridor. Compliance review runs alongside the build, so it never blocks launch at the end.




SE = solutions engineer, who owns the connection. The bold name leads; the others support.

## API review & credentials ›

Ten questions for the partner's API, and how each credential is exchanged and rotated.

### Partner API review questions

1. Is the payout API synchronous, asynchronous with a callback, or both? Which signal is final?
2. What is the full status model? Which states are final, and can a final state still change (returns, reversals)?
3. Which field carries idempotency, how long is it honoured, and what does a duplicate return?
4. Is there a status enquiry by our reference, and what are its rate limits?
5. How are callbacks signed and retried? Is ordering guaranteed? Which IP ranges send them?
6. Is account or alias validation available before sending, and does it return the account name?
7. What are the per-transaction and daily limits, batch windows, cutoffs and holidays?
8. What is the full error list? ISO 20022 reason codes or proprietary, and which are final?
9. How do prefunding, settlement and the settlement report work, and when do they arrive?
10. What screening does the partner run, and how does a compliance hold or reject reach us?

### Credential exchange

| METHOD | USED FOR | ROTATION |
|---|---|---|
| **OAuth 2.0** | Client-credentials token for API calls; cache until expiry | Rotate the client secret on a schedule |
| **mTLS** | Both sides prove identity at the transport layer | New certificate before expiry, old and new both trusted for an overlap |
| **JWT** | Signed requests or assertions; key chosen by `kid` | Publish the new key first, then switch signing |
| **API key** | Simple or legacy endpoints | Rotate; never in logs or URLs |
| **Webhook HMAC** | Proves a callback came from the partner | Two secrets valid during rotation |

CRYPTOGRAPHY PITFALLS I CHECK FOR

- ✓ AES-256-GCM authenticates the data; CBC does not, so it needs a separate MAC and is open to padding-oracle mistakes.
- ✓ Hex versus Base64: agree the encoding of keys, IVs and signatures; a mismatch looks like "invalid signature".
- ✓ Sign and verify the raw body bytes, not re-serialised JSON, and include a timestamp to stop replays.
- ✓ RSA for signatures or key wrapping, with the padding scheme written into the spec.

## Field & error mapping SAMPLE ›

Our fields to ISO 20022, and every partner error code to final, retryable, hold or unknown.

## Field mapping

Our canonical fields, the matching ISO 20022 pacs.008 element, and the check that runs before anything is sent. The partner's own field names are added from their spec.

| OUR FIELD | ISO 20022 (PACS.008) | VALIDATION BEFORE SEND |
|---|---|---|
| **Payout reference** | `PmtId/EndToEndId` | Unique per payout, at most 35 characters; also the idempotency anchor |
| **Amount + currency** | `IntrBkSttlmAmt @Ccy` | ISO 4217 decimals; within the rail's per-transaction cap |
| **Beneficiary name** | `Cdtr/Nm` | Length and character set per rail; transliteration rules agreed |
| **Account or alias** | `CdtrAcct/Id` · `CdtrAcct/Prxy` | Per rail: IBAN mod-97, CLABE check digit, IFSC format, E.164 mobile |
| **Beneficiary bank** | `CdtrAgt/FinInstnId` | BIC or clearing-system member ID in the partner's bank list |
| **Sender details** | `Dbtr/Nm` · `Dbtr/PstlAdr` | Originator data the corridor's rules require, complete before send |
| **Purpose** | `Purp/Cd` | From the corridor's allowed list |

## Error code mapping

Partner codes here are invented examples; the reason codes in brackets are ISO 20022 external status reason codes.

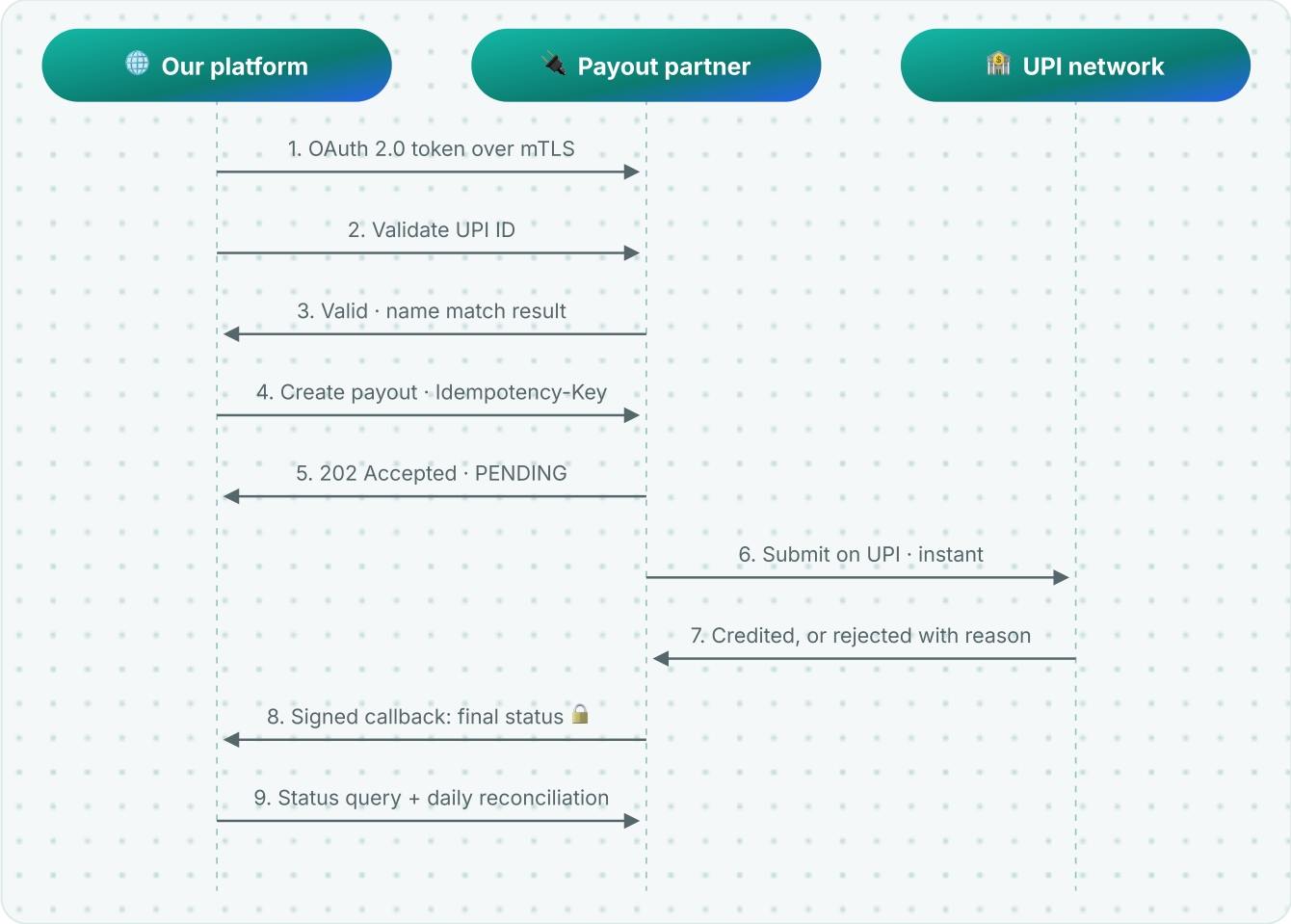
| PARTNER CODE | CANONICAL CODE | CLASS | WHAT WE DO |
|---|---|---|---|
| `P-101` **account not found** | `ACCOUNT_INVALID` (AC01) | Final | Ask the sender to correct the details; never retry as is |
| `P-102` **account closed** | `ACCOUNT_CLOSED` (AC04) | Final | Refund or new beneficiary details |
| `P-210` **name mismatch** | `NAME_MISMATCH` (BE01) | Final | Correct the name; track as a transliteration signal |
| `P-300` **over limit** | `LIMIT_EXCEEDED` (AM02) | Final | Route to a rail that allows the amount, if policy permits |
| `P-500` **partner timeout** | `TEMPORARY` | Retryable | Status enquiry first; retry only with the same idempotency key |
| `P-777` **compliance hold** | `COMPLIANCE_HOLD` (RR04) | Hold | Route to Compliance; never auto-retry or resend on another rail |
| **Any code not in the table** | `UNMAPPED` | Unknown | Treat as pending, query status, alert, then add the mapping |

How compliance shows up technically: a screening match or a name-matching hit usually arrives as a hold or a regulatory reject, not an error. It goes to the compliance queue with the evidence, and it is never treated as a retryable failure.

## Corridor blueprint (live demo) Interactive

Pick a rail to see the payout flow, and what to validate before money moves.

PAYOUT FLOW




**CLEARING**

Instant, 24×7

**BENEFICIARY ID**

UPI ID (VPA), e.g. name@handle

**VALIDATE BEFORE SEND**

UPI ID format, then a name lookup through the partner where offered

**WATCH FOR**

Limits set by NPCI and banks; a pending or deemed status needs a status check before any retry

🔒 Signed callback: the status we act on, after verifying it  Steps 2–3 run before money moves

### Rails primer: what I explain to engineers and business leads

**Clearing vs settlement**

Clearing exchanges and confirms the payment messages; settlement moves the funds between institutions. A "credited" status is not the same as settled.

**Instant vs deferred batch**

Instant rails confirm in seconds, all day. Batch rails clear in windows, so cutoffs and holidays decide the credit date.

**Alias vs account**

An alias (UPI ID, mobile number) is resolved to an account by the rail or a directory; validating it first avoids most rejections.

**ISO 20022 vs ISO 8583**

ISO 20022 (pacs.008 credit, pacs.002 status, camt.05x statements) for account rails; ISO 8583 (MTI 0200/0210, response code in field 39) for card-based payouts.

## Certification tests ›

Twelve cases every corridor passes before launch, with raw traces kept as evidence.

The test matrix every corridor passes before launch, with raw traces kept as evidence. Tick them off to try it.

0 / 12

- [ ] **Happy path on each rail**
  Credited, signed callback verified, reconciled
- [ ] **Invalid account format**
  Rejected by our validation, never sent
- [ ] **Account not found or closed**
  Mapped to AC01 / AC04, marked final
- [ ] **Name mismatch**
  Mapped, routed per policy
- [ ] **Amount over the rail cap**
  Rejected or re-routed, as the spec says
- [ ] **Same request, same idempotency key**
  Same payout returned; no second credit
- [ ] **Timeout on create**
  Status enquiry resolves it; no new payout
- [ ] **Repeated or out-of-order callbacks**
  Events deduplicated; a final status never goes backwards
- [ ] **Bad callback signature**
  Rejected and alerted; status unchanged
- [ ] **After cutoff or on a holiday**
  Correct expected credit date; not marked failed
- [ ] **Return after credit**
  Handled as a reversal; ledger updated
- [ ] **Key or certificate rotation**
  Old and new both work during the overlap

## Failure handling

Six production failures, how we detect each, and what we do instead of guessing.

What breaks in production, how we spot it, and what we do instead of guessing.

### Timeout on create

**DETECT**

No response, or a 5xx, after sending the payout

**FALLBACK**

The state is unknown, not failed. Query status by our reference; retry only with the same idempotency key if the partner confirms it never arrived.

### Repeated or out-of-order callbacks

**DETECT**

Same event again, or an older status after a newer one

**FALLBACK**

Deduplicate events, not the payout. Allow valid changes such as a return, never move a final status backwards, and reconcile with the status API.

### Rejection spike on one corridor

**DETECT**

Rejection rate for one reason code above its baseline

**FALLBACK**

Query the data, check our own changes first, then escalate with evidence. Pause or re-route if customers are affected.

### Unmapped error code

**DETECT**

A partner code with no entry in the mapping

**FALLBACK**

Hold as pending, query status, alert, and add the mapping with the partner's written meaning.

### Missed cutoff or holiday

**DETECT**

Batch payout submitted after the window

**FALLBACK**

Show the real expected credit date. Queue it; don't mark it failed or resend it.

### Compliance hold or sanctions match

**DETECT**

Hold status or a regulatory reject (RR04)

**FALLBACK**

Route to Compliance with the evidence. Never auto-retry or resend on another rail to get round it.

## Run & escalate

A SQL query that turns transaction data into an escalation, and what goes to the roadmap.

## From transaction data to an argument

Rejection rate by reason code, this week against the four weeks before. Table and column names are illustrative.

```sql
-- Rejection rate by reason code: this week vs the prior 4 weeks
WITH w AS (
  SELECT status, reason_code,
         created_at >= now() - interval '7 days' AS this_week
  FROM   payouts
  WHERE  partner_id = 'corvid' AND corridor = 'US-PH'
    AND  created_at >= now() - interval '35 days'
)
SELECT reason_code,
  round(100.0 * count(*) FILTER (WHERE this_week)
        / nullif((SELECT count(*) FROM w WHERE this_week), 0), 2)     AS pct_this_week,
  round(100.0 * count(*) FILTER (WHERE NOT this_week)
        / nullif((SELECT count(*) FROM w WHERE NOT this_week), 0), 2) AS pct_prior_4_weeks
FROM   w
WHERE  status = 'REJECTED'
GROUP BY reason_code
ORDER BY pct_this_week DESC;
```

## Escalation note ILLUSTRATIVE

**[Corvid Pay · US→PH · InstaPay] AC01 rejections 3.1% vs 0.4% baseline since 12 Nov**

**WHAT** Account-not-found rejections rose eight-fold on one rail; other rails are flat.

**EVIDENCE** Query output, 20 sample references with raw request and response traces, first and last timestamps.

**OUR SIDE** No mapping, validation or release changes since 5 Nov; the same accounts passed validation.

**IMPACT** Payouts rejected and refunded; customers asked to resend.

**ASK** Root cause by a set date, and a fix or written plan under the SLA's incident clause.

RECURRING FRICTION → PLATFORM ROADMAP

- ✓ Unmapped codes flagged automatically, not found in tickets.
- ✓ Checksums (IBAN, CLABE) and alias formats validated before send.
- ✓ Cutoff and holiday calendars per rail held in configuration.
- ✓ Certificate and key expiry monitored, with alerts well ahead.

## API kit ›

Sample Postman collection: token, validation, idempotent create, status and a signed callback.

**Postman collection** SAMPLE

Six requests for a fictional payout partner: OAuth token, validation, idempotent create, status and a signed callback.

- POST **Get OAuth 2.0 token** — client credentials; mTLS set in Postman's certificate settings
- POST **Validate account or alias** — returns the name for matching
- POST **Create payout** — with a fresh Idempotency-Key
- POST **Create it again, same key** — must return the same payout, not a new one
- GET **Payout status by our reference** — for timeouts and reconciliation
- POST **Replay a signed callback** — HMAC-SHA256 over the raw body, computed in a pre-request script

A sample to import into Postman. Base URLs, paths, field names and the callback body are placeholders to replace from the partner's spec.

## Key best practices

Eight rules for integration and ownership.

### INTEGRATION

**Unknown is not failed**
After a timeout, query status. Never create a second payout to "retry".

**One idempotency key per payout**
Reuse it on every retry, so a duplicate can never pay twice.

**Verify, then act**
Check the callback signature over the raw body before changing any status.

**Validate before money moves**
Checksums, alias formats and name checks catch most rejections up front.

### OWNERSHIP

**Escalate with data**
A query, sample traces and our own changes ruled out, in every escalation.

**Fix the cause**
A problem seen twice becomes a platform change or a roadmap item.

**Two audiences, one page**
Rails for engineers, contracts and SLAs for business leads, in the same note.

**Track every expiry**
Certificates, keys and secrets, with alerts long before they lapse.

## Rails I plan for

| | | | |
|---|---|---|---|
| **UPI**<br>India<br>instant | **IMPS**<br>India<br>instant | **InstaPay**<br>Philippines<br>instant | **PESONet**<br>Philippines<br>batch |
| **SPEI**<br>Mexico<br>instant | **NPSB / BEFTN**<br>Bangladesh<br>instant or batch | **Mobile money**<br>Kenya<br>instant | **SEPA**<br>Euro area<br>instant or next day |

Click a rail to see its payout flow.

Prepared by **Syed Aakhib** · LinkedIn

Proposed approach for a fictional payout partner, Corvid Pay; not a record of past payout-rail delivery. Partner codes, endpoints, field names and figures are placeholders or illustrative. Rail facts are summarised from public scheme information and must be confirmed against the partner's spec and scheme rules. Independent work sample; not affiliated with or endorsed by MoneyGram.